

Object Oriented Programming (OOP) Explained with Real-Life Analogies



Created by: Neeraj | [LinkedIn: neeraj-kumar1904](#)  | X: [@_19_neeraj](#)  | [GitHub: Neeraj05042001](#)  |

Things to Learn Today

Today we'll explore the fascinating world of Object-Oriented Programming (OOP) in JavaScript! We'll understand how programming concepts mirror real-world scenarios and learn the five fundamental principles that make code more organized, reusable, and maintainable.

Learning Objectives:

-  Understand Objects, Classes, and Functions
-  Master OOP principles with real-world analogies
-  Apply OOP concepts in JavaScript
-  Build maintainable and scalable code

Objects

Definition: An object is a collection of related data (properties) and functionality (methods) that represents a real-world entity or concept.

Creating Objects

1. Object Literal Syntax

```
const person = {  
  name: "John",  
  age: 25,  
  greet: function() {  
    return `Hello, I'm ${this.name}`;  
  }  
};
```

2. Constructor Function

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
  this.greet = function() {  
    return `Hello, I'm ${this.name}`;  
  };  
}  
  
const john = new Person("John", 25);
```

3. Object.create()

```
const personPrototype = {  
  greet: function() {  
    return `Hello, I'm ${this.name}`;  
  }  
};  
  
const person = Object.create(personPrototype);  
person.name = "John";  
person.age = 25;
```

☀ Real-Life Analogy

Think of a **smartphone** 📱:

- **Properties:** brand, model, color, battery percentage, storage
- **Methods:** makeCall(), sendMessage(), takePicture(), playMusic()



JavaScript Example

```
const smartphone = {  
  // Properties (data)  
  brand: "iPhone",  
  model: "15 Pro",  
  color: "Space Gray",  
  batteryPercentage: 85,  
  storage: "256GB",  
  
  // Methods (functionality)  
  makeCall: function(number) {  
    return `Calling ${number}...`;  
  },  
  
  sendMessage: function(contact, message) {
```

```

        return `Message sent to ${contact}: ${message}`;
    },

    takePicture: function() {
        return "📷 Picture taken!";
    }
};

console.log(smartphone.brand); // iPhone
console.log(smartphone.makeCall("123-456-7890")); // Calling 123-456-7890...

```

Knowledge Check

▼ Q: What's the difference between properties and methods in an object?

Answer:

- **Properties** are the data/characteristics of an object (like `brand` , `color`)
- **Methods** are the actions/functions that the object can perform (like `makeCall()` , `sendMessage()`)

▼ Q: What are the three ways to create objects in JavaScript?

1. **Object Literal Syntax:** `const obj = {}`
2. **Constructor Function:** `function Obj() {}` with `new Obj()`
3. **Object.create():** `Object.create(prototype)`



Classes

Definition: A class is a blueprint or template for creating objects with similar properties and methods. It defines the structure and behavior that objects of that type will have.

Real-Life Analogy

Think of a **house blueprint** 🏠:

- The blueprint defines: rooms, doors, windows, roof
- Each house built from this blueprint will have the same structure
- But each house can have different colors, furniture, or decorations

Class Syntax (ES6+)

```

class Person {
    // Constructor method
    constructor(name, age) {
        this.name = name;
    }
}

```

```

        this.age = age;
    }

    // Method
    greet() {
        return `Hello, I'm ${this.name}`;
    }

    // Static method
    static getSpecies() {
        return "Homo sapiens";
    }
}

// Creating instances
const john = new Person("John", 25);
const jane = new Person("Jane", 30);

```



JavaScript Example

```

class Car {
    // Constructor - runs when creating new object
    constructor(brand, model, color) {
        this.brand = brand;
        this.model = model;
        this.color = color;
        this.speed = 0;
    }

    // Methods
    startEngine() {
        return `${this.brand} ${this.model} engine started! 🚗`;
    }

    accelerate(increment) {
        this.speed += increment;
        return `Speed: ${this.speed} km/h`;
    }

    brake() {
        this.speed = 0;
        return "Car stopped! 🛑";
    }
}

// Creating objects from the class
const car1 = new Car("Toyota", "Camry", "White");
const car2 = new Car("Honda", "Civic", "Black");

```

```
console.log(car1.startEngine()); // Toyota Camry engine started! 🚗  
console.log(car1.accelerate(50)); // Speed: 50 km/h
```

🗨 Knowledge Check

▼ Q: What is a constructor in a class?

Answer: A constructor is a special method that runs automatically when a new object is created from a class. It's used to initialize the object's properties with values passed during object creation.

▼ Q: What is the difference between a class and an object?

- **Class:** A blueprint or template that defines the structure and behavior
- **Object:** An actual instance created from the class with specific values
- **Analogy:** Class is like a cookie cutter, Object is like the actual cookie

⚙ Functions

Definition: Functions are reusable blocks of code that perform specific tasks. In OOP context, functions become methods when they're part of objects or classes.

- **Function:** Standalone piece of code that performs a task.
- **Method:** A function that belongs to an object or class.

☀ Real-Life Analogy

Think of a **coffee machine** ☕:

- **Input:** Coffee beans, water, milk
- **Process:** Grind, heat, mix
- **Output:** Your favorite coffee

Types of Methods

1. Instance Methods

```
class Calculator {  
  add(a, b) {  
    return a + b; // Called on instances  
  }  
}
```

```
const calc = new Calculator();
calc.add(5, 3); // 8
```

2. Static Methods

```
class MathUtils {
  static multiply(a, b) {
    return a * b; // Called on the class itself
  }
}

MathUtils.multiply(4, 5); // 20
```

JavaScript Example

```
// Regular function
function calculateArea(length, width) {
  return length * width;
}

// Function as a method in an object
const calculator = {
  add: function(a, b) {
    return a + b;
  },

  multiply: function(a, b) {
    return a * b;
  },

  calculateCircleArea: function(radius) {
    return Math.PI * radius * radius;
  }
};

console.log(calculateArea(5, 3)); // 15
console.log(calculator.add(10, 5)); // 15
console.log(calculator.calculateCircleArea(3)); // 28.27
```

Object Oriented Programming (OOP)

Definition: OOP is a programming paradigm that organizes code around objects and classes rather than functions and logic. It helps create modular, reusable, and maintainable code by modeling real-world entities.

☀️ Why OOP is Important?

1. 🔄 **Reusability:** Write once, use multiple times
2. 🛡️ **Security:** Control access to data
3. 🧩 **Modularity:** Break complex problems into smaller parts
4. 🔧 **Maintainability:** Easy to update and debug
5. 👥 **Collaboration:** Multiple developers can work on different classes

💬 Knowledge Check: OOP Benefits

▼ Name three main benefits of using OOP?

1. **Code Reusability:** Classes can be reused to create multiple objects
2. **Better Organization:** Related data and functions are grouped together
3. **Easier Maintenance:** Changes to a class affect all instances automatically

💻 Real-World Structure Example

```
// City structure using OOP thinking
/*
City 🏙️
├── Houses 🏠
│   ├── Roof
│   ├── Doors
│   └── Windows
├── Cars 🚗
│   ├── Model
│   ├── Brand
│   ├── Color
│   └── Wheels
├── People 👥
├── Hospitals 🏥
└── Roads 🛣️
*/
```

🌍 Real-world Examples of OOP

1. 🎮 Game Development

```
class Player {
    constructor(name, health, level) {
        this.name = name;
        this.health = health;
        this.level = level;
    }
}
```

```

    }

    attack(enemy) {
        return `${this.name} attacks ${enemy.name}!`;
    }
}

class Enemy {
    constructor(type, health, damage) {
        this.type = type;
        this.health = health;
        this.damage = damage;
    }
}

```

2. 🛒 E-commerce Website

```

class User {
    constructor(name, email) {
        this.name = name;
        this.email = email;
        this.cart = [];
    }

    addToCart(product) {
        this.cart.push(product);
    }
}

class Product {
    constructor(name, price, category) {
        this.name = name;
        this.price = price;
        this.category = category;
    }
}

```

3. 🏠 Banking Software

```

class BankAccount {
    constructor(accountNumber, ownerName, initialBalance = 0) {
        this.accountNumber = accountNumber;
        this.ownerName = ownerName;
        this.balance = initialBalance;
    }

    deposit(amount) {
        if (amount > 0) {
            this.balance += amount;
        }
    }
}

```



```

        return `Deposited ${amount}. New balance: ${this.balance}`;
    }
    return "Invalid amount";
}

withdraw(amount) {
    if (amount > 0 && amount <= this.balance) {
        this.balance -= amount;
        return `Withdrew ${amount}. New balance: ${this.balance}`;
    }
    return "Invalid amount or insufficient funds";
}

getBalance() {
    return this.balance;
}
}

```

Knowledge Check: Real-world Examples

▼ How would you model a "Library Book" using OOP concepts?

```

class LibraryBook {
    constructor(title, author, isbn) {
        this.title = title;
        this.author = author;
        this.isbn = isbn;
        this.isAvailable = true;
        this.borrower = null;
    }

    borrow(borrowerName) {
        if (this.isAvailable) {
            this.isAvailable = false;
            this.borrower = borrowerName;
            return `Book borrowed by ${borrowerName}`;
        }
        return "Book is not available";
    }

    return() {
        this.isAvailable = true;
        this.borrower = null;
        return "Book returned successfully";
    }
}

```

The Five Pillars of OOP

1. Abstraction

Definition: Abstraction means hiding complex implementation details and showing only the essential features of an object. It's like using a TV remote - you don't need to know how it works internally, just which buttons to press.

Real-Life Analogy

When you drive a car , you use:

- **Steering wheel** to turn
- **Accelerator** to speed up
- **Brake** to slow down

You don't need to know how the engine combusts fuel or how the transmission works!

JavaScript Example

```
class ATM {
  #bankConnection; // Private property
  #securitySystem; // Private property

  constructor() {
    this.#bankConnection = "Connected to Bank Server";
    this.#securitySystem = "Active";
  }

  // Public interface - user only sees these methods
  withdrawMoney(amount, pin) {
    if (this.#validatePin(pin)) {
      return this.#processWithdrawal(amount);
    }
    return "Invalid PIN";
  }

  checkBalance(pin) {
    if (this.#validatePin(pin)) {
      return "Your balance is $1,500";
    }
    return "Invalid PIN";
  }

  // Private methods - hidden from user
  #validatePin(pin) {
    // Complex validation logic hidden
    return pin === "1234";
  }
}
```

```
}

#processWithdrawal(amount) {
  // Complex banking logic hidden
  return `Dispensing ${amount}`;
}

const atm = new ATM();
console.log(atm.withdrawMoney(100, "1234")); // Dispensing $100
// atm.#validatePin("1234"); // Error! Private method not accessible
```

Knowledge Check

▼ Q: What's the benefit of abstraction in programming?

Answer: Abstraction simplifies complex systems by:

- Hiding implementation details
- Providing a simple interface to interact with
- Reducing complexity for the user
- Making code more maintainable

Example: You don't need to know how `Array.sort()` works internally, just how to use it.

2. Encapsulation

Definition:

Encapsulation is the bundling of data and methods that operate on that data within a single unit (class), and restricting access to some of the object's components. It's like a capsule that protects the medicine inside.

Real-Life Analogy

Think of a **medicine capsule** :

- The active ingredients are protected inside
- You can't directly access or modify the medicine
- You interact with it through a controlled interface (swallowing it)

JavaScript Example

```

class BankAccount {
  #balance; // Private property
  #pin; // Private property

  constructor(initialBalance, pin) {
    this.#balance = initialBalance;
    this.#pin = pin;
    this.accountNumber = Math.random().toString().substr(2, 8);
  }

  // Public methods - controlled access
  deposit(amount) {
    if (amount > 0) {
      this.#balance += amount;
      return `Deposited ${amount}. New balance: ${this.#balance}`;
    }
    return "Invalid amount";
  }

  withdraw(amount, pin) {
    if (this.#validatePin(pin) && amount <= this.#balance && amount > 0) {
      this.#balance -= amount;
      return `Withdrawn ${amount}. Remaining balance: ${this.#balance}`;
    }
    return "Transaction failed";
  }

  getBalance(pin) {
    if (this.#validatePin(pin)) {
      return `Your balance is ${this.#balance}`;
    }
    return "Access denied";
  }

  // Private method
  #validatePin(pin) {
    return pin === this.#pin;
  }
}

const account = new BankAccount(1000, "1234");
console.log(account.deposit(500)); // Deposited $500. New balance: $1500
console.log(account.getBalance("1234")); // Your balance is $1500
// console.log(account.#balance); // Error! Cannot access private property

```

Benefits of Encapsulation

1. **Data Protection:** Prevents unauthorized access
2. **Controlled Access:** Data can only be modified through defined methods
3. **Maintainability:** Internal implementation can change without affecting external code

4. **Validation:** Can validate data before setting it

Knowledge Check

▼ Q: How does encapsulation improve security?

Answer: Encapsulation prevents direct access to sensitive data by making properties private. External code can only interact with the object through public methods, which can include validation and security checks.

▼ How do you create private fields in modern JavaScript?

Use the `#` symbol before the field name:

```
class MyClass {  
  #privateField = "secret";  
  
  getPrivateField() {  
    return this.#privateField;  
  }  
}
```

3. Inheritance

Definition: Inheritance allows one class (child) to inherit properties and methods from another class (parent), promoting code reuse and establishing relationships between classes.

Real-Life Analogy

Think of **family traits** :

- Children inherit characteristics from parents (eye color, height)
- They also develop their own unique traits
- The family shares common behaviors but each member is unique

JavaScript Example

```
// Parent class  
class Vehicle {  
  constructor(brand, model, year) {  
    this.brand = brand;  
    this.model = model;  
    this.year = year;  
    this.speed = 0;  
  }  
}
```

```

    start() {
        return `${this.brand} ${this.model} is starting...`;
    }

    accelerate(increment) {
        this.speed += increment;
        return `Speed: ${this.speed} km/h`;
    }

    getInfo() {
        return `${this.year} ${this.brand} ${this.model}`;
    }
}

// Child class - inherits from Vehicle
class Car extends Vehicle {
    constructor(brand, model, year, doors) {
        super(brand, model, year); // Call parent constructor
        this.doors = doors;
        this.fuelType = "Gasoline";
    }

    // Override parent method
    start() {
        return `${super.start()} 🚗 Car engine roaring!`;
    }

    // New method specific to Car
    openTrunk() {
        return "Trunk opened! 🚗";
    }
}

// Another child class
class Motorcycle extends Vehicle {
    constructor(brand, model, year, engineSize) {
        super(brand, model, year);
        this.engineSize = engineSize;
        this.hasWindshield = false;
    }

    start() {
        return `${super.start()} 🏍️ Motorcycle revving!`;
    }

    wheelie() {
        return "Performing wheelie! 🏍️";
    }
}

const myCar = new Car("Toyota", "Camry", 2023, 4);
const myBike = new Motorcycle("Harley", "Sportster", 2022, "883cc");

```

```
console.log(myCar.start()); // Toyota Camry is starting... 🚗 Car engine roaring!
console.log(myBike.start()); // Harley Sportster is starting... 🏍️ Motorcycle revving!
console.log(myCar.getInfo()); // 2023 Toyota Camry (inherited method)
```

🧠 Knowledge Check

▼ Q: What's the difference between `super()` and `this` in inheritance?

Answer:

- `super()` calls the parent class constructor or methods
- `this` refers to the current object instance
- `super()` must be called before using `this` in a child constructor

4. 🎭 Polymorphism

Definition: Polymorphism means "many forms" - it allows objects of different classes to be treated as the same type, but behave differently based on their specific implementation.

☀️ Real-Life Analogy

Think of musical instruments 🎵:

- All can "play music" but each sounds different
- Piano plays music differently than guitar
- Same action ("play"), different results

💻 JavaScript Example

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  makeSound() {
    return "Some generic animal sound";
  }

  move() {
    return `${this.name} is moving`;
  }
}

class Dog extends Animal {
```

```

    makeSound() {
        return `${this.name} says: Woof! Woof! 🐶`;
    }

    move() {
        return `${this.name} is running on four legs 🐶`;
    }
}

class Cat extends Animal {
    makeSound() {
        return `${this.name} says: Meow! Meow! 🐱`;
    }

    move() {
        return `${this.name} is gracefully walking 🐱`;
    }
}

class Bird extends Animal {
    makeSound() {
        return `${this.name} says: Tweet! Tweet! 🐦`;
    }

    move() {
        return `${this.name} is flying high 🐦`;
    }
}

// Polymorphism in action
const animals = [
    new Dog("Buddy"),
    new Cat("Whiskers"),
    new Bird("Tweety")
];

// Same method call, different behaviors
animals.forEach(animal => {
    console.log(animal.makeSound());
    console.log(animal.move());
    console.log("---");
});

/* Output:
Buddy says: Woof! Woof! 🐶
Buddy is running on four legs 🐶
---
Whiskers says: Meow! Meow! 🐱
Whiskers is gracefully walking 🐱
---
Tweety says: Tweet! Tweet! 🐦
Tweety is flying high 🐦

```


*/

Knowledge Check

▼ Q: How does polymorphism make code more flexible?

Answer: Polymorphism allows you to write code that works with the parent class but automatically uses the correct child class behavior. You can add new classes without changing existing code that uses them.

▼ Q: What is the main advantage of polymorphism?

Polymorphism allows:

- Writing code that works with objects of multiple types
- Adding new types without changing existing code
- Treating different objects uniformly through a common interface
- More flexible and extensible code design

5. Composition

Definition: Composition is an alternative to inheritance where you build complex objects by combining simpler ones. It follows the principle "has-a" rather than "is-a".

Real-Life Analogy

Think of a **computer** :

- Has-a CPU, RAM, hard drive, keyboard, monitor
- Each component can be replaced independently
- Computer is composed of these parts working together

JavaScript Example

```
// Component classes
class Engine {
  constructor(type, horsepower) {
    this.type = type;
    this.horsepower = horsepower;
  }

  start() {
    return `${this.type} engine started (${this.horsepower} HP)`;
  }
}
```

```

    }
}

class GPS {
    constructor(brand) {
        this.brand = brand;
    }

    navigate(destination) {
        return `${this.brand} GPS: Navigating to ${destination}`;
    }
}

class MusicSystem {
    constructor(brand) {
        this.brand = brand;
        this.currentSong = "No song playing";
    }

    playMusic(song) {
        this.currentSong = song;
        return `🎵 Now playing: ${song} on ${this.brand} system`;
    }
}

// Main class using composition
class ModernCar {
    constructor(brand, model) {
        this.brand = brand;
        this.model = model;

        // Composition - "has-a" relationship
        this.engine = new Engine("V6", 300);
        this.gps = new GPS("Garmin");
        this.musicSystem = new MusicSystem("Bose");
    }

    startCar() {
        return this.engine.start();
    }

    goTo(destination) {
        return this.gps.navigate(destination);
    }

    playMusic(song) {
        return this.musicSystem.playMusic(song);
    }

    getCarInfo() {
        return `${this.brand} ${this.model} with ${this.engine.type} engine`;
    }
}

```

```
const myCar = new ModernCar("BMW", "X5");
console.log(myCar.startCar()); // V6 engine started (300 HP)
console.log(myCar.goTo("New York")); // Garmin GPS: Navigating to New York
console.log(myCar.playMusic("Bohemian Rhapsody")); // 🎵 Now playing: Bohemian Rhapsody on Bos
```

Benefits of Composition 🎯

1. **Flexibility:** Can mix and match behaviors
2. **Avoid Deep Inheritance:** No complex inheritance hierarchies
3. **Runtime Changes:** Can add/remove behaviors dynamically
4. **Better Testing:** Individual components can be tested separately

🧠 Knowledge Check

▼ Q: When should you use composition over inheritance?

Answer: Use composition when objects have a "has-a" relationship rather than "is-a". Composition is more flexible because you can change components independently and avoid complex inheritance hierarchies.

🔄 Quick Recap

📋 The Five OOP Principles Summary:

1. 🏰 **Abstraction** → Hide complexity, expose interface
2. 📦 **Encapsulation** → Bundle + protect state
3. 👥 **Inheritance** → Share logic across classes
4. 🦊 **Polymorphism** → Same method, different behaviors
5. 🧱 **Composition** → Build by combining units

💡 Key Relationships:

- **Inheritance:** "is-a" relationship (Car **is-a** Vehicle)
- **Composition:** "has-a" relationship (Car **has-a** Engine)

🎤 Common Interview Questions

▼ Q1: What are the main advantages of OOP?

Answer:

1. **Reusability:** Code can be reused through inheritance and composition
2. **Modularity:** Complex problems broken into smaller, manageable parts
3. **Maintainability:** Easier to update and debug
4. **Security:** Encapsulation protects data
5. **Scalability:** Easy to add new features without affecting existing code

▼ Q2: Explain the difference between inheritance and composition.

Answer:

- **Inheritance (is-a):** Child class inherits from parent class. Creates tight coupling.
- **Composition (has-a):** Class contains objects of other classes. Creates loose coupling and is more flexible.

Example:

- Inheritance: `Dog extends Animal` (Dog is-a Animal)
- Composition: `Car has Engine` (Car has-a Engine)

▼ Q3: What is method overriding in JavaScript?

Answer: Method overriding is when a child class provides a different implementation of a method that exists in the parent class. The child's version "overrides" the parent's version.

```
class Parent {  
  greet() { return "Hello from Parent"; }  
}  
  
class Child extends Parent {  
  greet() { return "Hello from Child"; } // Overrides parent method  
}
```

▼ Q4: How do you achieve private properties in JavaScript classes?

Answer: Use the `#` symbol before property or method names to make them private:

```
class MyClass {  
  #privateProperty = "secret";  
  
  #privateMethod() {  
    return "private";  
  }  
  
  publicMethod() {  
    return this.#privateProperty; // Can access private members internally  
  }  
}
```

```
}  
}
```

▼ Q5: What is the purpose of the `super` keyword?

Answer: The `super` keyword is used to:

1. Call the parent class constructor: `super()`
2. Access parent class methods: `super.methodName()`
3. Must be called before using `this` in child constructor

```
class Child extends Parent {  
    constructor(name) {  
        super(); // Call parent constructor  
        this.name = name;  
    }  
  
    method() {  
        return super.method() + " additional text";  
    }  
}
```

Practice Tasks

Task 1: Basic Class Creation

Create a `Book` class with properties: title, author, pages, isRead. Add methods to mark as read and get book info.

► Solution

Task 2: Inheritance Challenge

Create a `Person` class and extend it with `Student` and `Teacher` classes. Each should have unique methods.

► Solution

Task 3: Encapsulation Practice

Create a `BankAccount` class with private balance and methods for deposit, withdraw, and balance inquiry.

► Solution

Task 4: Polymorphism Implementation

Create different shape classes that all have a `calculateArea()` method but implement it differently.

► Solution

Task 5: Composition Challenge

Create a `Computer` class that uses composition with `CPU`, `RAM`, and `Storage` components.

► Solution

Task 6: Real-World Application

Create a simple library management system using OOP principles.

► Solution

Task 7: Advanced OOP - Abstract Classes Pattern

Create an abstract-like class structure for different types of employees.

► Solution

Task 8: Design Pattern - Factory Pattern

Implement a factory pattern for creating different types of vehicles.

► Solution

Final Challenge Project

Task 9: Mini E-commerce System

Create a complete mini e-commerce system using all OOP principles.

► Solution

Congratulations!

You've successfully completed Day 29 of your JavaScript journey! 🎉

What You've Mastered Today:

-  Understanding Objects, Classes, and Functions
-  Five Core OOP Principles (Abstraction, Encapsulation, Inheritance, Polymorphism, Composition)
-  Real-world applications of OOP concepts
-  Advanced patterns and best practices
-  Building complex systems using OOP

 Next Steps:

- Practice building more complex applications using OOP
- Explore design patterns (Factory, Observer, Singleton)
- Learn about modules and code organization
- Study advanced JavaScript concepts like Prototypes and Prototype Chain

Remember: OOP is not just about syntax—it's about thinking in terms of objects and their relationships. Keep practicing with real-world examples! 💪

 Quick Reference Card:

Concept	Key Points	When to Use
Abstraction	Hide complexity, show interface	Complex systems with simple user interactions
Encapsulation	Bundle data + methods, restrict access	Protecting sensitive data and maintaining state
Inheritance	Share code between classes (is-a)	When classes have common behavior
Polymorphism	Same interface, different implementations	When you need flexible, extensible code
Composition	Build objects from other objects (has-a)	When you need flexible relationships



Let's Connect & Level Up Together

Follow me for daily JavaScript tips, insightful notes, and project-based learning.

 [X \(Twitter\)](#)

 [LinkedIn](#)

 [Telegram](#)

 [Threads](#)

 Dive into the full notes on GitHub → [40 Days JavaScript](#)

© 2025 • Crafted with  by [Neeraj](#)

 *Next: Day - 30: From Zero to OOP Hero with JavaScript ES6 Classes* 

 Happy JavaScript coding! 

Happy Coding!  Keep building amazing things with JavaScript! 